SOFTWARE DESIGN DOCUMENT

GuardianAI

Real-Time Automated Fall Detection and Alert System

For Elderly and Patient Safety in Low-Resource Environments

Group ID	FYP-2026-G6
Document	Software Design Document (SDD / SDS)
Version	1.0
Deliverable	FYP-I Checkpoint-2
Programme	BS Computer Science

Team Member	Roll No	Role
Kashan M. Ashraf	BSCS-01007	Video Processing + Testing
Umer Ahmed	BSCS-01011	AI Model + Development
Imran	BSCS-01014	UI Design + Documentation

Repository: FYP-2026-G6-RealTimeAutomatedFallDetectionAndAlertSystem

Prepared for review and approval by the Faculty Supervisor

Document Control

Revision History

Version	Date	Author	Description of Change
0.1	Checkpoint-1	FYP-2026-G6	Initial architecture sketch submitted as part of the SRS (Section 2.1).
1.0	Checkpoint-2	FYP-2026-G6	First complete Software Design Document. Documents the as-built design of the working MVP: layered architecture, component decomposition, class model, data design, interface contracts, behavioural models, and algorithm specifications. Records the migration from MediaPipe to YOLOv8-Pose and the addition of the hybrid decision engine.

Document Approval

Role	Name	Signature / Date
Group Lead	Kashan M. Ashraf	
Faculty Supervisor		

Related Documents

Document	Location	Relationship
Software Requirements Specification (SRS) v2	`/docs`	Parent document. This SDD realises the functional requirements FR-1 to FR-5 defined in the SRS (see Traceability Matrix, Section 11).
FYP Proposal	`/reports`, `/presentations`	Defines the original problem statement and project objectives.
Source Code	`/src`	The implementation described by this design.

Table of Contents

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the software architecture and detailed design of **GuardianAI**, a real-time automated fall detection and alert system. Where the Software Requirements Specification (SRS) states *what* the system must do, this document specifies *how* the system is constructed to satisfy those requirements: how responsibility is divided across modules, how data is represented and transformed as it moves through the pipeline, how components interact at runtime, and which algorithms implement the detection logic.

The design presented here is **as-built** rather than aspirational. Every component, interface, and algorithm documented in Sections 3 to 9 corresponds to working code in the project repository and was verified against a running instance of the system during the preparation of this document. Features that remain planned for FYP-2 are explicitly separated in Section 12 so that the boundary between delivered and future work is unambiguous.

1.2 Scope

This document covers the complete GuardianAI edge application, comprising:

- **Video ingestion and pose estimation** — acquiring frames from a webcam or video file and extracting human skeletal keypoints using YOLOv8-Pose.
- **Biomechanical feature engineering** — deriving interpretable physical quantities (body orientation angle, head drop speed, ground proximity, aspect ratio) from raw keypoints.
- **Hybrid decision engine** — fusing an explainable rule-based state machine with a temporal deep learning classifier (CNN-LSTM) to classify activity as `normal`, `warning`, or `fall`.
- **Multi-channel alerting** — console output, an on-screen popup, an audible buzzer, native operating-system notifications, optional SMTP email, and persistent JSON event logging with JPEG frame captures.
- **Web monitoring dashboard** — a Flask-served single-page interface streaming annotated MJPEG video alongside live telemetry and an incident history.
- **Offline training subsystem** — dataset preprocessing, synthetic data generation, model training, and ONNX export.

Out of scope for this document: clinical validation methodology, hospital network integration, mobile application design, and cloud deployment topology. These are addressed in the FYP-2 planning material referenced in Section 12.

1.3 Intended Audience

Reader	How to use this document
Faculty Supervisor / Evaluator	Sections 3, 8 and 9 give the architectural rationale and algorithmic detail. Section 11 traces each SRS requirement to its implementing component. Section 12 states honestly what is complete and what is not.
Project Team Members	Sections 5 and 6 are the working reference for module responsibilities, class interfaces, and data structures when extending the system.
External Developer / Maintainer	Sections 3.5, 7 and 13 provide the technology stack, the public API contract, and every tunable configuration parameter.

1.4 Definitions and Acronyms

Term	Definition
YOLOv8-Pose	A single-stage convolutional neural network from Ultralytics that simultaneously detects persons and regresses 17 skeletal keypoints per person.
Keypoint	A detected body joint, represented as a triple `(x, y, confidence)`.
CNN-LSTM	A hybrid architecture in which a 1-D convolutional front-end extracts local spatial-temporal patterns and an LSTM models longer-range temporal dependencies.
Sequence buffer	A fixed-length sliding window (30 frames) of feature vectors forming one input sample to the temporal classifier.
Decision fusion	The weighted combination of a rule-based verdict and a neural network verdict into a single classification.
Temporal smoothing	A majority-vote filter over recent frame classifications that suppresses single-frame flicker.
XAI	Explainable Artificial Intelligence — design practice ensuring an automated decision can be traced to human-interpretable evidence.
MJPEG	Motion JPEG; a streaming format delivering a sequence of independent JPEG images over one HTTP response.
HUD	Heads-Up Display — the telemetry overlay drawn onto the annotated video frame.
Edge deployment	Running inference locally on the machine attached to the camera, rather than transmitting video to a remote server.

1.5 References

1. Software Requirements Specification (SRS) v2 — GuardianAI, FYP-2026-G6, `/docs/SRS_document_V2.docx`.
2. IEEE Std 1016-2009, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*.
3. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
4. Ultralytics YOLOv8 Documentation — `https://docs.ultralytics.com`.
5. PyTorch Documentation — `https://pytorch.org/docs`.
6. Flask Documentation — `https://flask.palletsprojects.com`.

2. Design Considerations

2.1 Assumptions and Dependencies

Assumption / Dependency	Design Implication
Single fixed camera with an unobstructed view of the monitored area	The design uses a monocular 2-D pipeline. No depth sensing or multi-camera fusion is required, which keeps hardware cost low — a core project objective.
Adequate ambient lighting for body-outline detection	No infrared or low-light enhancement stage is included. Detection confidence thresholds (0.5) assume normally lit indoor scenes.
Commodity PC hardware, GPU optional	The `yolov8n` (nano) variant is selected over larger models, and the device is resolved at runtime (`"auto"` → CUDA if available, otherwise CPU).
Python ≥ 3.8 runtime with PyTorch, OpenCV, Ultralytics and Flask available	Dependencies are pinned by lower bound in `requirements.txt` ; no compiled extensions are authored by the team.
Operator is present at the monitoring station or on the same machine	Alerting is designed for local delivery first (buzzer, popup, OS notification). Remote channels (email, SMS) are secondary and optional.

2.2 Design Constraints

Constraint	Consequence for the Design
Real-time latency. A fall must be recognised within 1–2 seconds.	The pipeline is strictly per-frame and single-pass. The temporal window is bounded at 30 frames, and all blocking side-effects (buzzer, popup, e-mail, OS notification) are dispatched on daemon threads so they can never stall frame processing.
Explainability. Clinical settings require an auditable reason for every alert.	Rule-based biomechanics are retained as a first-class decision path rather than being replaced by the neural network. Raw angle and speed values are surfaced on the HUD, in the `/status` API and on the dashboard.
Privacy. Continuous video of vulnerable people is highly sensitive.	Video is processed in memory and never recorded. Only a single JPEG frame is persisted per confirmed alert, into a local log directory.
Graceful degradation. The system must not fail hard.	A missing model checkpoint downgrades to pure rule-based operation instead of raising; a camera read failure triggers reconnection rather than terminating the stream.
Academic scope. FYP-1 targets a demonstrable MVP.	Persistence uses flat JSON files rather than a database server; authentication and multi-tenant concerns are deferred to FYP-2.

2.3 Design Goals and Guiding Principles

Principle	How it is applied
Separation of concerns	Each pipeline stage is an independent module with a single responsibility: perception (`pose_estimator`), representation (`feature_extractor`), inference (`fall_detector_model`), orchestration (`inference`), notification (`alert_system`), and delivery (`api_server`).
Centralised configuration	Every tunable threshold, path and hyper-parameter lives in typed dataclasses in `config.py` . No magic numbers are embedded in algorithm code, so behaviour can be retuned without touching logic.
Substitutability	Three interchangeable temporal architectures (LSTM, CNN-LSTM, Transformer) are produced by a single factory function selected by a configuration string, enabling comparative evaluation without code changes.
Fail-safe defaults	Absent model weights, an unavailable sound device, a missing SMTP configuration, and a disconnected camera all degrade to a reduced but functioning system.
Observability	The system continuously exposes its internal biomechanical state — not merely its verdict — through the HUD, the REST API, and structured event logs.

3. System Architecture

3.1 Architectural Style

GuardianAI combines two complementary architectural styles:

- A **layered pipe-and-filter pipeline** governs the detection path. Each video frame flows unidirectionally through a fixed sequence of transformations — capture → pose estimation → feature extraction → classification → decision fusion → alerting — where every stage consumes the output of its predecessor and has no knowledge of stages further downstream. This style suits continuous stream processing and makes each stage independently testable and replaceable.
- A **client–server** layer wraps the pipeline for delivery. A Flask application exposes the live annotated stream and the system state over HTTP, allowing any browser on the network to act as a thin monitoring client with no installation.

The composite is deployed as a single **edge application**: capture, inference and alerting all execute on the machine physically attached to the camera, so no video ever leaves the premises.

3.2 Layered Architecture

The system is organised into five horizontal layers. Dependencies point strictly downward: a layer may use the services of the layer beneath it, but never the reverse. The Configuration module is a cross-cutting concern consumed by every layer.

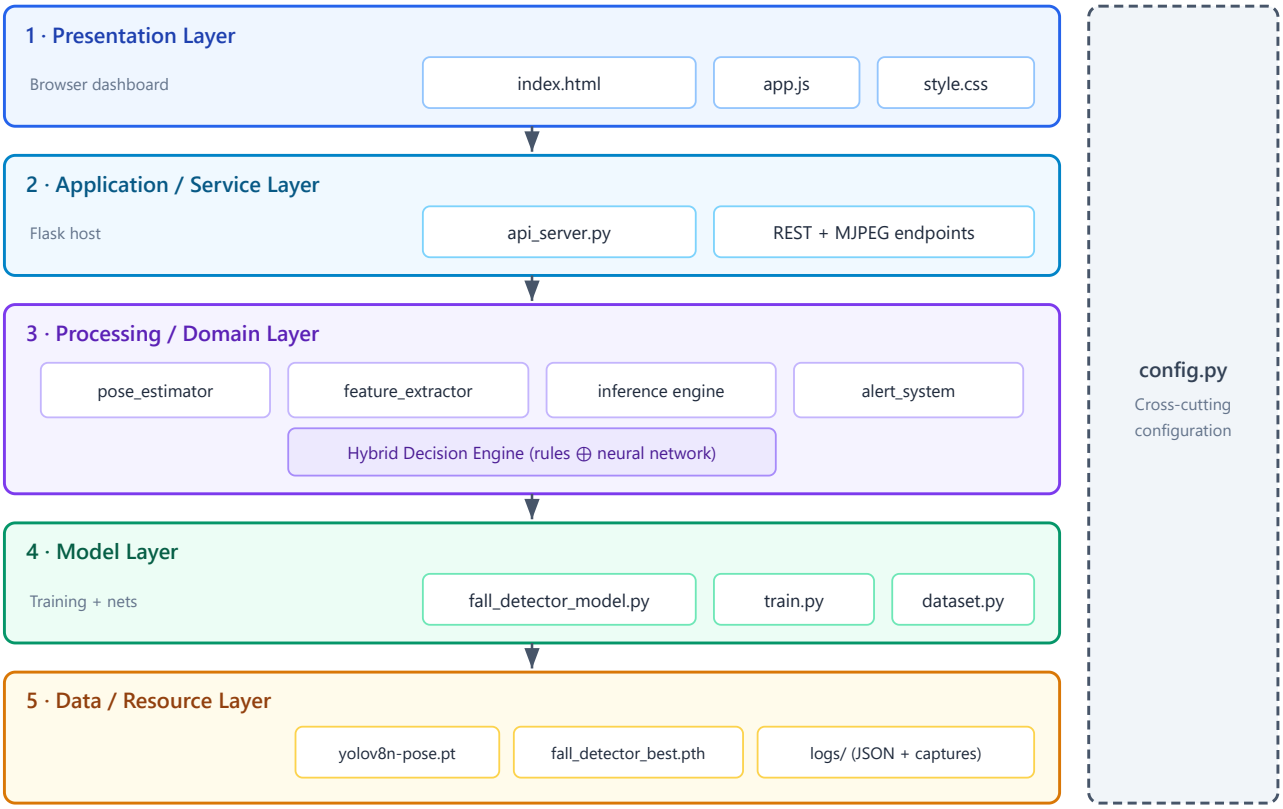

Figure 3.1 — Layered architecture. Dependencies flow downward only; `config.py` is consumed by all layers.

3.3 Component Decomposition

Figure 3.2 shows the runtime components and the direction of their dependencies. The `FallDetectionInference` class is the orchestrator: it owns one instance each of the pose estimator, feature extractor, neural model and alert system, and is the only component that knows the full pipeline order.

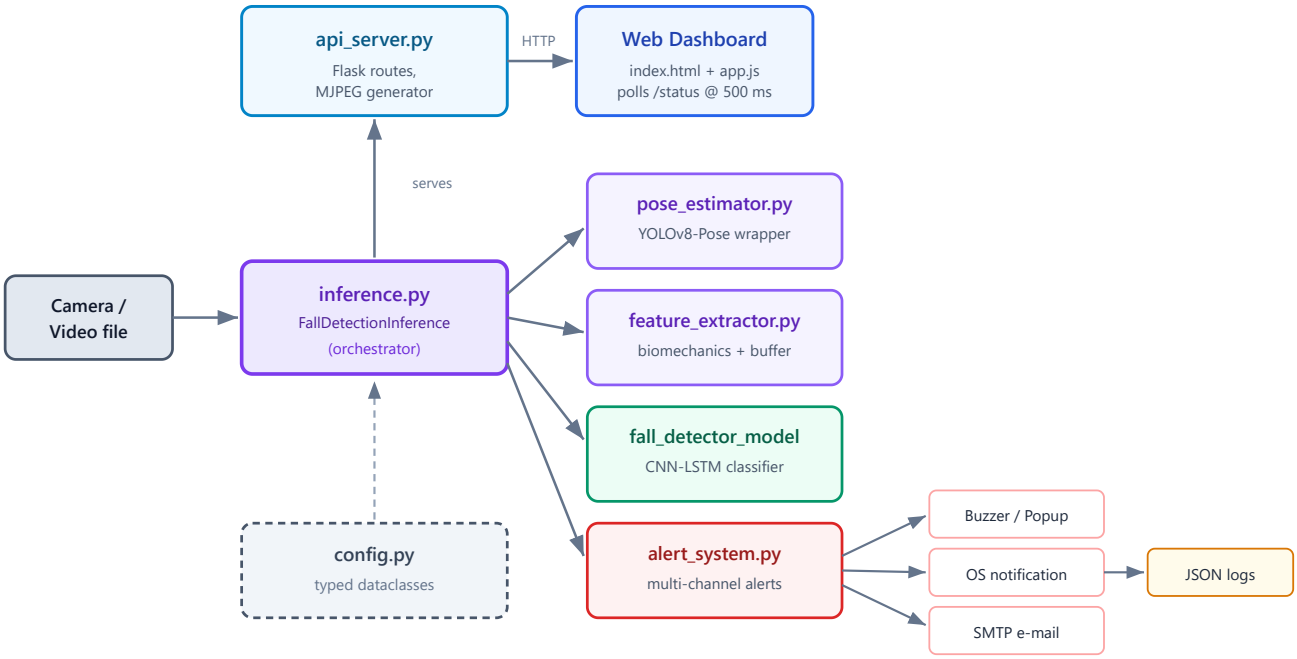

Figure 3.2 — Component decomposition and dependency directions. Solid arrows denote runtime invocation; the dashed arrow denotes configuration injection.

3.4 Deployment Architecture

GuardianAI deploys as a single process on one physical node. Figure 3.3 shows the deployment view, including the trust boundary that keeps raw video inside the premises.

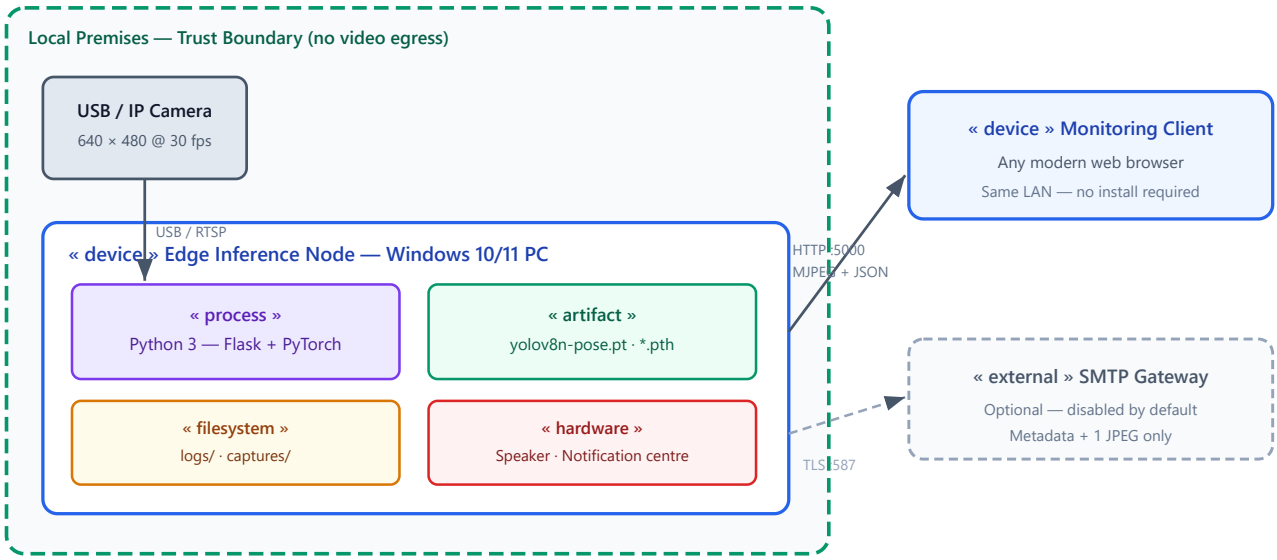

Figure 3.3 — Deployment view. Raw video never crosses the trust boundary; only alert metadata and a single JPEG may leave via the optional SMTP path.

3.5 Technology Stack

Layer	Technology	Design Justification
Pose estimation	Ultralytics YOLOv8-Pose (nano)	Single-stage detection and keypoint regression in one pass. The nano variant runs acceptably on CPU, and unlike the originally planned MediaPipe it natively supports multi-person scenes and is more robust to partial occlusion.
Numerical / CV	OpenCV, NumPy	Industry-standard frame capture, JPEG encoding and drawing primitives; vectorised array maths for feature computation.
Deep learning	PyTorch	Dynamic graphs simplify experimentation across the three temporal architectures; mature ONNX export path for future edge optimisation.
Web service	Flask	Minimal footprint appropriate for an edge device. Native support for streaming responses, which the MJPEG feed depends on.
Front-end	HTML5, CSS3, vanilla JavaScript	Deliberately framework-free: no build step, no `node_modules`, and the dashboard is directly inspectable by evaluators.
Notifications	`winsound`, `plyer`, `smtplib`	Layered channels: an immediate local buzzer, a native OS toast that is visible when the dashboard is closed, and optional remote e-mail.
Persistence	JSON files + JPEG captures	Zero-configuration, human-readable and directly reviewable during a demonstration — sufficient for FYP-1 scope. A relational store is deferred to FYP-2.

4. Data Flow and Processing Pipeline

4.1 Data Flow Diagram

Figure 4.1 traces a single video frame through the complete pipeline, annotating the data representation at every hand-off point. This transformation chain executes once per frame, at approximately 25–30 frames per second.

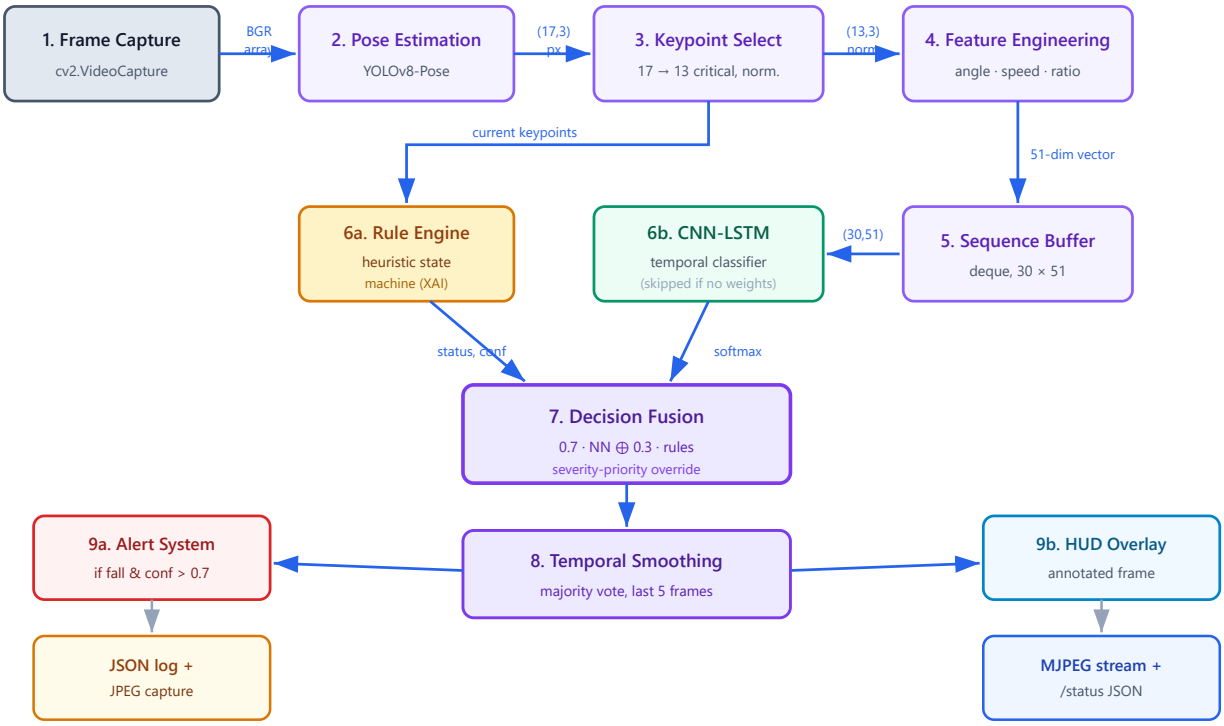

Figure 4.1 — Per-frame data flow. Note the two parallel decision paths (6a rules, 6b neural network) converging at the fusion stage; the rule path operates on the current frame while the neural path requires a full 30-frame window.

4.2 Per-Frame Processing Walkthrough

#	Stage	Input → Output	Behaviour
1	Frame capture	Camera → BGR `ndarray`	OpenCV reads one frame. On failure the reconnection strategy of Section 10.2 is invoked.
2	Pose estimation	BGR frame → `(N,17,3)`	YOLOv8-Pose infers person instances and keypoints. Detections below confidence 0.5 are discarded. The single-person estimator takes the first (highest-scoring) instance.
3	Keypoint selection and normalisation	`(17,3)` px → `(13,3)` normalised	The 13 keypoints listed in `CRITICAL_KEYPOINTS` are retained; `x` and `y` are divided by frame width and height so downstream logic is resolution-independent.
4	Feature engineering	`(13,3)` → 51-dim vector	39 flattened raw values are concatenated with 12 engineered features (Section 6.2).
5	Sequence buffering	51-dim → `(30,51)` or `None`	A bounded `deque` accumulates vectors. Until 30 frames are present the neural path is skipped and the system operates on rules alone.
6a	Rule engine	`(13,3)` → (status, confidence)	Deterministic biomechanical state machine (Section 9.4). Always available, including at start-up.
6b	Neural inference	`(30,51)` → 3-class softmax	Executed under `torch.no_grad()`. Skipped entirely when no checkpoint is loaded.
7	Decision fusion	two verdicts → one verdict	Weighted blend with a severity-priority override (Section 9.5).
8	Temporal smoothing	verdict → stabilised verdict	Majority vote across the last five frames suppresses single-frame flicker.
9a	Alerting	verdict → side effects	Triggered only when status is `fall` *and* confidence exceeds 0.7, then further gated by the cooldown.
9b	Presentation	verdict → annotated frame	HUD overlay drawn, frame JPEG-encoded and yielded to the MJPEG stream; telemetry cached for `/status`.

5. Detailed Component Design

5.1 Configuration Subsystem — `config.py`

All system parameters are declared as typed Python dataclasses aggregated by a single `SystemConfig` root object. A module-level singleton `CONFIG` is instantiated at import time and shared by default, while every component also accepts an injected configuration instance — a design that supports unit testing and per-experiment overrides without global mutation.

Dataclass	Responsibility
`PoseConfig`	Detection and tracking confidence thresholds; the `CRITICAL_KEYPOINTS` index list selecting 13 of the 17 YOLO joints.
`FeatureConfig`	Sequence length, stride, feature dimensionality, and the rule-based thresholds for speed, angle, aspect ratio and stillness.
`ModelConfig`	Architecture selector, layer sizes, dropout, and all training hyper-parameters including class weights for the imbalanced fall class.
`CameraConfig`	Capture source index or path, resolution and target frame rate.
`AlertConfig`	Per-channel enable flags, SMTP settings, cooldown period, popup duration and log directory.
`SystemConfig`	Root aggregate. Its `__post_init__` hook rewrites every relative path to an absolute path anchored at the project directory, so the application behaves identically regardless of the working directory from which it is launched. `get_device()` resolves `"auto"` to CUDA or CPU at runtime.

5.2 Pose Estimation Module — `pose_estimator.py`

This module encapsulates all coupling to the Ultralytics library, presenting the rest of the system with a stable interface that returns plain NumPy arrays. Two classes are provided.

5.2.1 `PoseEstimator` (single subject)

- `process_frame(frame) → (keypoints | None, annotated_frame)` — runs inference, draws the skeleton overlay, and returns the 13 selected keypoints in normalised coordinates.
- **Occlusion resilience.** When no person is detected, `_handle_lost_tracking()` returns the last known keypoints for up to `_max_lost_frames = 10` consecutive frames before reporting loss. This prevents momentary detection dropouts — common when a subject turns or is briefly obscured — from resetting the temporal buffer and discarding an in-progress fall sequence.
- **Visualisation.** `_draw_custom_skeleton()` renders a 14-edge skeleton using a two-pass stroke (a dark outer line beneath a light inner line) that remains legible against both bright and dark backgrounds. Joints below confidence 0.4 are not drawn.

5.2.2 `MultiPersonPoseEstimator`

Returns a list of keypoint arrays for up to `max_persons` (default 5) detected subjects, with per-person colour cycling in the overlay. The class is implemented and exercised by the `fyp2_multi_demo.py` demonstration script; integrating it into the main dashboard pipeline — which requires per-person feature buffers and independent state machines — is scheduled for FYP-2 (Section 12).

5.3 Feature Extraction Module — `feature_extractor.py`

Converts geometric keypoints into biomechanically meaningful quantities and maintains the temporal window. Two public entry points serve the two decision paths:

- `update(keypoints) → sequence | None` — appends the 51-dimensional vector for the current frame to the bounded buffer and returns the full `(30, 51)` window once it is saturated. When `keypoints` is `None` a

zero vector is appended, so the temporal spacing of the buffer remains physically correct across detection gaps.

- `get_rule_based_status(keypoints) → (status, confidence)` — evaluates the explainable state machine on the current frame and updates the persistent fall and lying counters.

The module maintains two *independent* previous-head-position variables, `_prev_head_y` (used for the neural feature vector) and `_prev_head_y_rule` (used by the rule engine). Keeping them separate ensures the rule path remains deterministic and auditable even though both derive from the same underlying measurement. The most recent angle and speed are cached in `_last_angle` and `_last_speed` for telemetry display.

> **Coordinate convention.** Rule thresholds are expressed in pixels against a fixed *virtual* 640 × 480 frame. Normalised keypoints are multiplied back into this virtual space before the rules are applied, so a single set of thresholds behaves consistently across cameras of differing native resolution.

A companion class, `TemporalFeatureProcessor`, supports the offline training path: it builds overlapping sequences with a stride of 5 frames, assigns each sequence a label by majority vote, and computes the mean and standard deviation used for feature standardisation.

5.4 Temporal Classification Models — `fall_detector_model.py`

Three interchangeable PyTorch architectures share the identical tensor contract `(batch, 30, 51) → (batch, 3)` and are produced by the `create_model()` factory, which dispatches on `ModelConfig.architecture`. All three terminate in an attention or CLS-token pooling stage rather than naive last-hidden-state extraction, so the classifier can weight the frames that actually carry the fall signature.

Architecture	Structure	Design trade-off
`FallDetectorLSTM`	2-layer bidirectional LSTM (hidden 128) → additive attention pooling → 3-layer MLP head.	Baseline. Models temporal order well but treats the 51 input dimensions as an unstructured vector, ignoring spatial relationships between joints.
`FallDetectorCNNLSTM` **(default)**	1-D CNN stack (64 then 128 channels, kernel 3, each with BatchNorm + ReLU + dropout) applied across the temporal axis → bidirectional LSTM → attention pooling → MLP head with BatchNorm.	**Selected.** The convolutional front-end learns local joint-motion patterns before the LSTM models longer-range progression, giving the best accuracy-per-millisecond balance for edge hardware.
`FallDetectorTransformer`	Linear projection + LayerNorm → sinusoidal positional encoding → 2 encoder layers (4 heads, GELU) with a prepended CLS token → classification head.	Strongest long-range modelling via self-attention, but higher compute cost and a greater tendency to overfit the modest keypoint dataset available at FYP-1 scale.

The module additionally provides `count_parameters()` for model-size reporting and `export_to_onnx()`, which traces the network with a dummy `(1, 30, 51)` input and a dynamic batch axis, producing an opset-13 graph for future runtime-optimised deployment.

5.5 Hybrid Decision Engine — `inference.py`

`FallDetectionInference` is the pipeline orchestrator and the only class aware of the end-to-end sequence. It composes the pose estimator, feature extractor, neural model and alert system, and owns the smoothing history.

Model loading with graceful fallback

`_load_model()` implements a deliberate resolution order: it first probes for `fall_detector_best.pth` — the checkpoint written by the training loop at its best validation epoch — in the directory of the configured model path, falling back to the configured filename itself. If no checkpoint exists, or if loading raises for any reason (architecture mismatch, corrupt file), the method logs a warning and returns `None`. The system then runs in pure rule-based mode. This satisfies the reliability requirement that a missing model must never produce an uncaught exception, and it is what allows the MVP to be demonstrated on a machine that has never run training.

Frame processing

`process_frame()` executes stages 2–9 of Figure 4.1 and returns a dictionary containing the status, confidence percentage, person count and the annotated frame. It also refreshes `_current_summary`, the telemetry snapshot served by the `/status` endpoint. The alert system is invoked only when the smoothed status is `fall` *and* confidence exceeds `SystemConfig.confidence_threshold` (0.7), providing a final gate above the fusion logic.

Heads-up display

`_draw_hud()` composites three overlay groups onto the frame: a status box (colour-coded green, amber or red), a telemetry panel showing body angle, drop speed and a derived stability bar, and a rolling FPS counter computed from a 30-sample timestamp window. Rendering the raw biomechanical values — not merely the verdict — directly onto the video is the primary Explainable-AI affordance of the system.

5.6 Alert and Notification Subsystem — `alert_system.py`

`AlertSystem` centralises all outbound notification. A single public method, `trigger_alert(status, confidence, frame, person_id)`, fans out to every enabled channel and returns a boolean indicating whether the alert fired or was suppressed by the cooldown.

Channel	Execution	Design notes
Console	Synchronous	ANSI colour-coded banner. Negligible cost; kept inline so it is guaranteed to appear before any threaded channel.
On-screen popup	Daemon thread	A full-colour OpenCV window displayed for `popup_duration_ms` (default 3000 ms) then destroyed. Threaded because `cv2.waitKey()` blocks for the full duration.
Audible buzzer	Daemon thread	A three-tone pattern (1000 Hz, 1500 Hz, 1000 Hz at 200 ms each) via `winsound` on Windows, degrading to terminal bells elsewhere. Threaded because `winsound.Beep()` blocks for roughly 600 ms — long enough to visibly stall the video pipeline if executed inline.
OS notification	Daemon thread	A native desktop toast raised through `plyer`, ensuring an alert is seen even when the browser dashboard is closed or the operator is working in another application. Failures are caught and logged rather than propagated.
E-mail (optional)	Daemon thread	HTML message with the captured JPEG attached, sent over SMTP with STARTTLS. Disabled by default; silently skipped when no sender or recipient is configured.
JSON event log	Synchronous	Appends a structured record to the current day's log file and rewrites it atomically. Kept synchronous so the audit trail cannot lose an event to a race.

Alert governance

- **Cooldown.** A single `_last_alert_time` gate enforces `alert_cooldown_seconds` (default 30 s) across *all* channels collectively, preventing a sustained fall state from generating a continuous stream of notifications.
- **Mute.** `set_muted()` suppresses the buzzer, popup and OS notification while deliberately leaving console output and JSON logging active — so muting the demonstration environment never compromises the evidentiary record.
- **Frame capture.** When a frame is supplied, a JPEG is written to `logs/captures/fall_<id>_<HHMMSS>.jpg` and its path is recorded in the event.
- **Log continuity.** On construction the system reloads the current day's log file, so restarting the application preserves rather than truncates the day's event history.

5.7 Web API and Dashboard — `api_server.py`, `static/`

The Flask application exposes the pipeline over HTTP. The detector is created lazily by `get_detector()` on first use, so process start-up is fast and model loading is deferred until a client actually connects.

- **Streaming.** `generate_frames()` is a Python generator yielding `multipart/x-mixed-replace` JPEG parts. Each iteration performs one full detection cycle, so the stream and the detection loop are the same loop — there is no duplicated inference.
- **Camera resilience.** Read failures increment a counter and retry briefly; after 15 consecutive failures the capture handle is released, the process pauses one second, and the device is reopened. A `finally` block guarantees the handle is released when a client disconnects, so a paused-then-resumed feed can always reacquire the device.

- **Concurrency.** The server runs with `threaded=True` so that the long-lived MJPEG response cannot starve the concurrent `/status` polling requests. The Werkzeug auto-reloader is explicitly disabled (`use_reloader=False`) because reloader restarts tear down the camera handle and model mid-stream.
- **Front-end.** `app.js` polls `/status` every 500 ms and updates the status card, telemetry tiles, event list and connection indicators. The video element consumes the MJPEG endpoint directly, so no frame data passes through JavaScript.

5.8 Training and Dataset Subsystem — `train.py`, `dataset.py`

This subsystem is offline: it produces the checkpoint the inference path consumes, and does not execute during live monitoring.

- `dataset.py` defines the canonical label mapping (`normal`=0, `warning`=1, `fall`=2), extracts frames from source videos at a target frame rate, runs them through the same `PoseEstimator` and `FeatureExtractor` used at inference time — guaranteeing train/serve feature parity — and emits PyTorch `Dataset` objects. A `WeightedRandomSampler` compensates for the natural scarcity of fall examples.
- `train.py` implements the training loop with class-weighted cross-entropy (`[1.0, 2.0, 3.0]`, penalising missed falls most heavily), learning-rate scheduling on plateau, early stopping with patience 15, best-checkpoint retention, and ONNX export.
- A synthetic data generator allows the full pipeline to be exercised end-to-end without a labelled video corpus, which is what makes `python main.py demo` runnable on any machine.

5.9 Class Model

Figure 5.1 shows the principal classes, their key members, and their composition relationships.

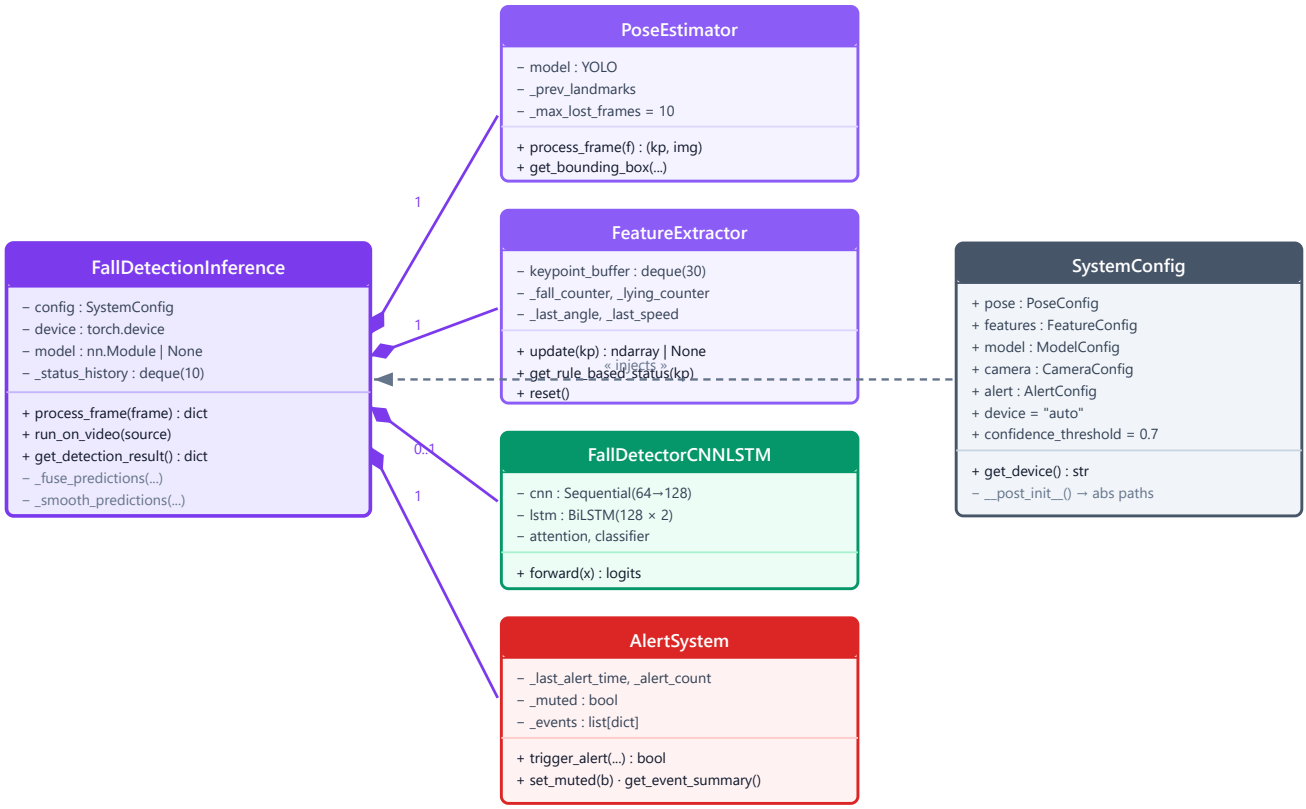

Figure 5.1 — Class model. Filled diamonds denote composition: `FallDetectionInference` owns the lifecycle of each collaborator. The neural model has multiplicity 0..1, reflecting the rule-only fallback mode.

6. Data Design

6.1 Keypoint Representation

YOLOv8-Pose emits 17 COCO-format keypoints per detected person. GuardianAI retains the 13 that carry fall-relevant information, discarding the four facial landmarks (eyes and ears) which add noise without contributing to postural analysis.

Array index	YOLO index	Joint	Role in detection
0	0	Nose	Head reference point for orientation angle, drop speed and ground proximity.
1–2	5, 6	Left / right shoulder	Upper-torso extent; contributes to bounding-box aspect ratio.
3–4	7, 8	Left / right elbow	Limb extent.
5–6	9, 10	Left / right wrist	Limb extent; bracing gestures.
7–8	11, 12	Left / right hip	**Torso centre.** The midpoint of these two joints is the lower anchor of the body-orientation vector (Section 9.1).
9–10	13, 14	Left / right knee	Lower-body posture.
11–12	15, 16	Left / right ankle	Ground contact reference; vertical extent.

Each keypoint is stored as `[x, y, confidence]` with `x` and `y` normalised to `[0, 1]` against frame width and height, giving a `(13, 3)` `float32` array per person per frame.

6.2 Feature Vector Composition

Each frame is encoded as a 51-dimensional vector formed by concatenating the flattened raw keypoints with the engineered biomechanical features.

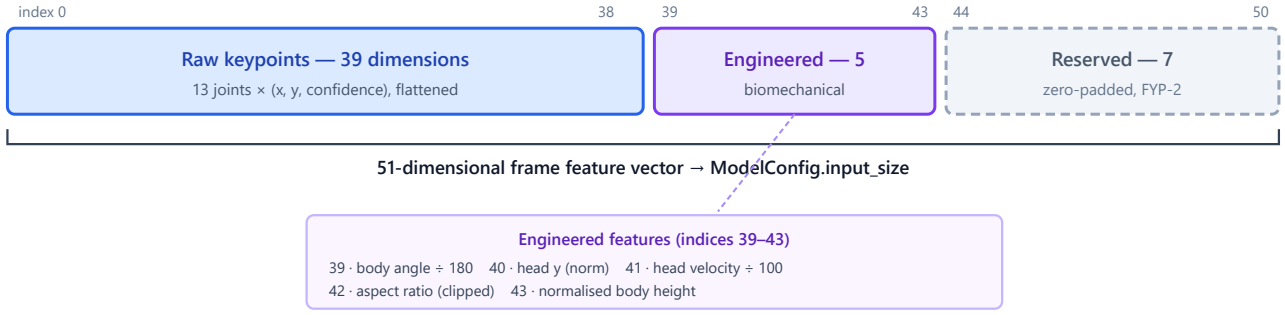

Figure 6.1 — Feature vector layout. Seven dimensions are intentionally reserved and zero-filled so additional engineered features can be introduced in FYP-2 without altering the network input shape or invalidating existing checkpoints.

6.3 Temporal Sequence Buffer

Frame vectors accumulate in a `collections.deque` bounded at `sequence_length = 30`, representing roughly one second of motion at 30 fps. The bounded deque provides O(1) append with automatic eviction of the oldest frame, which is exactly the sliding-window semantics required and avoids any explicit array shifting.

- **Warm-up.** Until 30 frames are buffered, `update()` returns `None` and the neural path is bypassed; the rule engine carries detection alone.
- **Gap handling.** Frames with no detected person contribute a zero vector rather than being skipped, preserving the real-time spacing of the window.
- **Training stride.** Offline sequence generation advances by `stride = 5` frames, producing overlapping windows that multiply the effective dataset size while retaining temporal diversity.

6.4 Event Log Schema

Events are appended to a per-day file `logs/fall_events_YYYYMMDD.json` containing a JSON array of records. Daily partitioning keeps files small enough to open and review during a demonstration and gives a natural retention boundary.

```
[
  {
    "status":     "fall",              // "warning" | "fall"
    "confidence": 94.3,                // percentage, 2 dp
    "timestamp":  "2026-07-26T00:32:33.915502",   // ISO-8601, local
    "person_id":  0                    // reserved for multi-person (FYP-2)
  }
]
```

The richer in-memory event record additionally carries `event_id` and `frame_capture` (the JPEG path). The persisted schema is deliberately narrower — a stable, minimal, machine-readable contract for downstream

analysis.

6.5 Persistent Artefacts

Artefact	Format	Purpose and lifecycle
`yolov8n-pose.pt`	PyTorch weights	Pre-trained pose model, read-only, shipped with the project.
`models/fall_detector_best.pth`	PyTorch checkpoint	Best-validation-epoch weights written by training; auto-discovered at inference.
`models/fall_detector.onnx`	ONNX graph	Optional export for optimised runtimes. Not required by the live pipeline.
`logs/fall_events_*.json`	JSON array	Append-only audit trail, one file per day.
`logs/captures/*.jpg`	JPEG	One frame per confirmed alert — the only video-derived data written to disk.
`data/processed/*.npz`	NumPy archive	Preprocessed training sequences and normalisation statistics.

> **Privacy by design.** No continuous video is recorded at any point. The only imagery that reaches persistent storage is a single JPEG per alert event, retained locally so that a caregiver can verify the incident.

7. Interface Design

7.1 REST API Contract

The Flask application exposes the following HTTP interface on port 5000. All JSON responses use `application/json`; error conditions return an `{"error": "..."}` body with an appropriate status code.

Method	Route	Purpose	Response
GET	`/`	Serve the dashboard single-page application.	`text/html`
GET	`/static/<path>`	Serve CSS and JavaScript assets.	Static file
GET	`/video_feed`	Continuous annotated video stream. Each part is a fully processed frame with skeleton and HUD.	`multipart/x-mixed-replace; boundary=frame`
GET	`/health`	Liveness probe; reports whether the neural checkpoint loaded.	`{status, model_loaded, timestamp}`
GET	`/status`	Primary telemetry endpoint, polled by the dashboard every 500 ms.	`{current_status, event_summary, camera_connected, alerts_muted}`
POST	`/detect`	Stateless single-frame inference. Accepts `multipart/form-data` with an `image` file, or JSON with a base64 `image` field.	`{status, confidence, keypoints_detected, processing_time_ms}`
POST	`/detect/batch`	Sequence inference over an ordered list of base64 frames, enabling true temporal classification on uploaded clips.	`{results[], final_status, final_confidence, num_frames, processing_time_ms}`
POST	`/alerts/mute`	Toggle or explicitly set alert muting. Body `{}` toggles; `{"muted": bool}` sets.	`{muted}`
POST	`/reset`	Clear the feature buffer and smoothing history without restarting the process.	`{message}`

Representative `/status` payload

```
{
  "current_status": {
    "status":       "normal",     // normal | warning | fall
    "confidence":   100.0,        // percentage
    "person_count": 1,
    "angle":        87.5,         // degrees — XAI evidence
    "speed":        0.0           // px/frame — XAI evidence
  },
  "event_summary": {
    "total_events": 3,
    "falls":        1,
    "warnings":     2,
    "log_file":     "logs/fall_events_20260726.json"
  },
  "camera_connected": true,
  "alerts_muted":     false
}
```

The `angle` and `speed` fields are the explainability contract: any client can display the physical evidence behind a verdict without re-deriving it. The `camera_connected` and `alerts_muted` fields let the dashboard reflect true system state rather than optimistic hard-coded indicators.

7.2 User Interface Design

The dashboard is a single responsive page using a two-column layout: a primary video region and a secondary telemetry column, over a persistent left navigation rail.

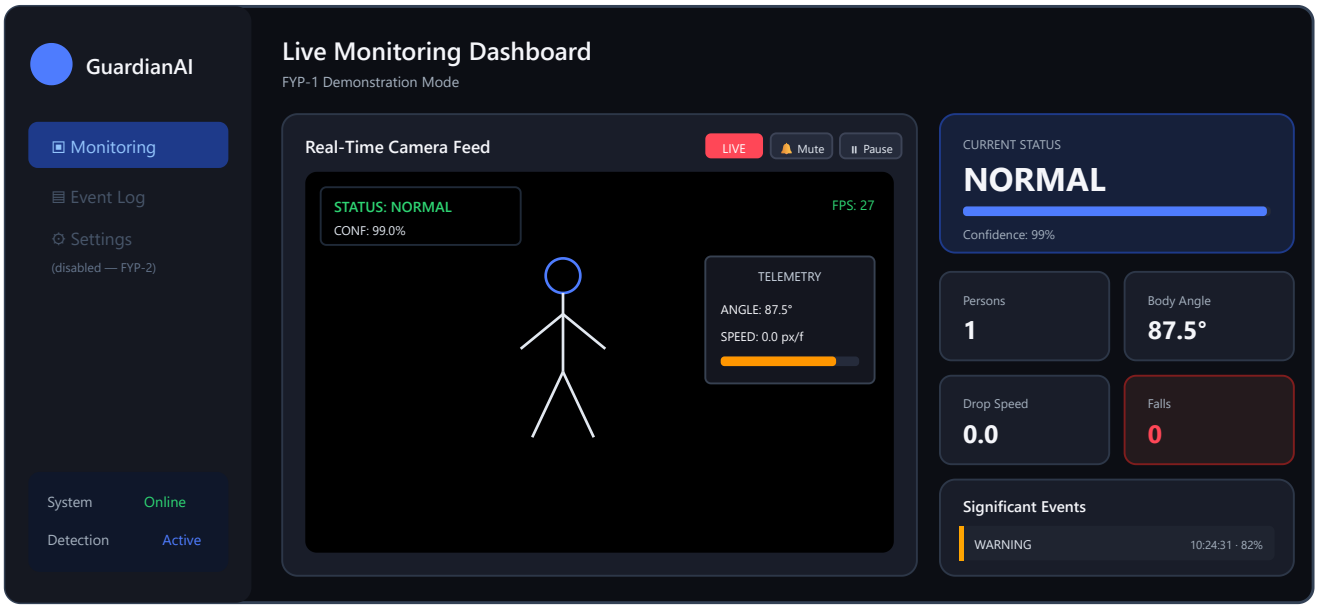

Figure 7.1 — Dashboard layout (Monitoring view). Telemetry appears twice by design: burned into the video frame as the HUD, and as DOM elements driven by `/status` polling.

UI element	Behaviour and design rationale
Status card	Displays the smoothed verdict with a confidence bar. Background gradient shifts blue → amber → red with severity, giving a pre-attentive signal readable across a room.
Telemetry tiles	Persons, body angle, drop speed and cumulative falls, refreshed twice per second from `/status`.
Pause / Resume	Detaches the `<img>` source, closing the MJPEG connection. Because detection runs inside the streaming generator, this halts processing and therefore all alerts — the mechanism by which a demonstration can be silenced instantly.
Mute / Unmute	Calls `/alerts/mute`. Suppresses buzzer, popup and OS notification server-side while video and logging continue, so the operator can observe detection without audible disruption.
System / Detection indicators	Bound to real state: *System* reflects reachability of the API; *Detection* shows Active, Paused or No Camera derived from `camera_connected`.
Significant Events	Client-side rolling list (capped at 20) appended on every transition away from `normal`.
Event Log / Settings	Presented but disabled, with a tooltip marking them as FYP-2 scope — an intentional signal of planned scope rather than a broken control.

7.3 Hardware Interface

Interface	Specification
Camera	Any device enumerable by OpenCV — integrated webcam, USB camera, or IP/RTSP stream addressed by URL. Default source index 0, requested at 640 × 480 @ 30 fps.
Audio output	System default device, driven through `winsound.Beep()` on Windows. No dedicated sound hardware or audio file is required.
Compute	x86-64 CPU minimum. A CUDA-capable GPU is detected and used automatically when present, but is not required.
Network	Standard TCP/IP. Port 5000 for the dashboard; outbound 587 only if e-mail alerting is enabled.

8. Behavioural Design

8.1 Detection State Machine

The rule engine maintains an explicit three-state model with hysteresis provided by two persistent counters. Counters — rather than instantaneous thresholds — are what prevent a single noisy frame from producing an alert.

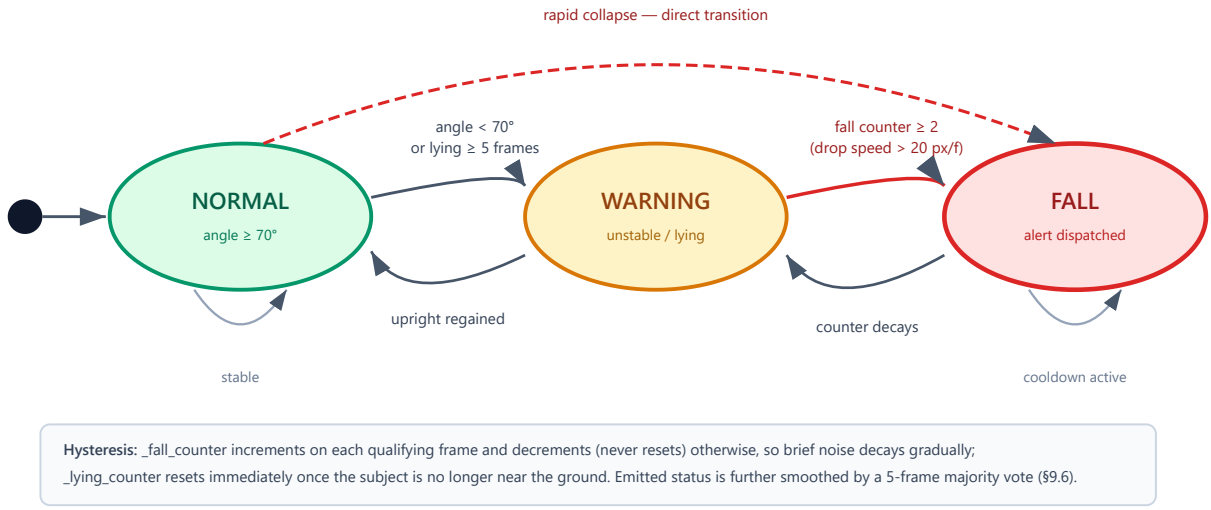

Figure 8.1 — Rule-engine state machine. The dashed edge shows that a sufficiently rapid collapse can transition directly from NORMAL to FALL without passing through WARNING.

8.2 Fall Detection and Alert Sequence

Figure 8.2 traces the interaction between components for a single frame in which a fall is confirmed.

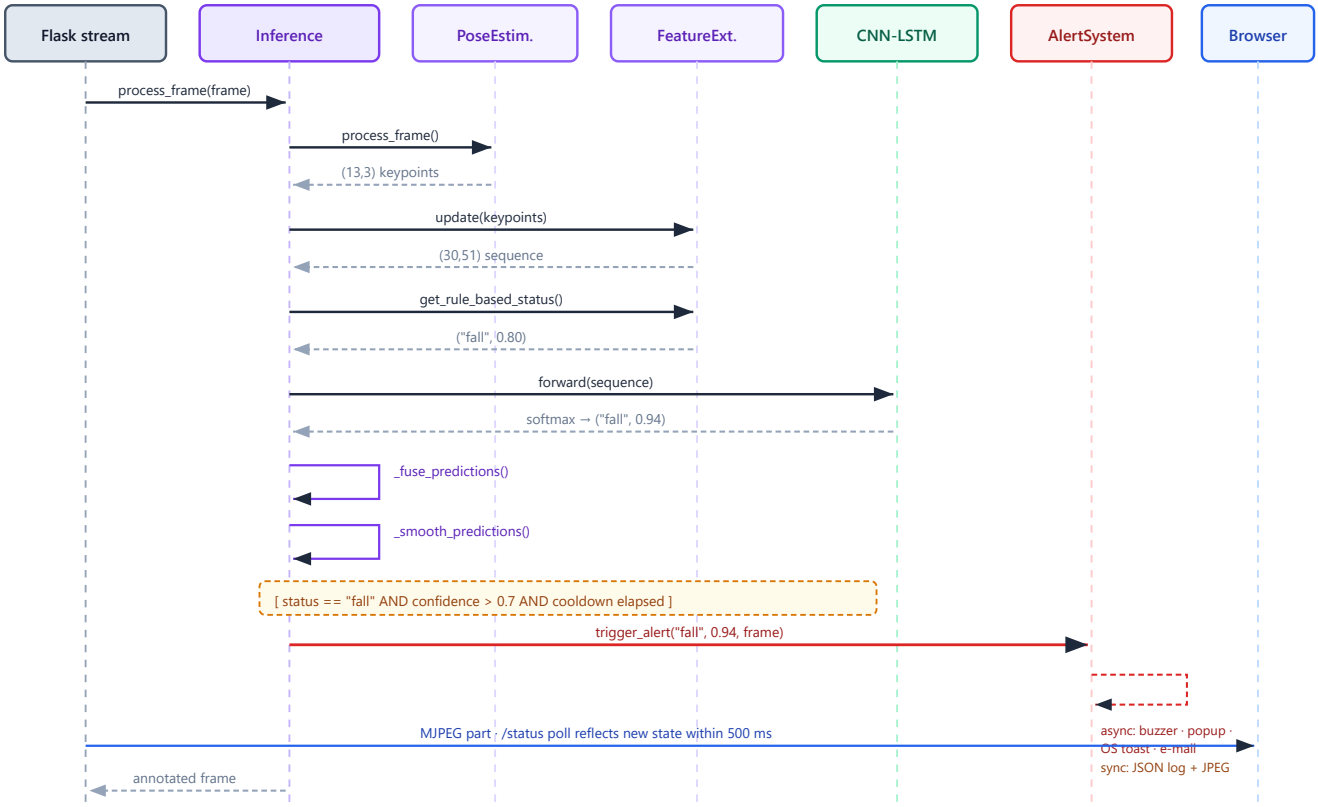

Figure 8.2 — Fall detection sequence. Solid arrows are synchronous calls; dashed arrows are returns. All blocking alert channels are dispatched asynchronously so the frame loop is never stalled.

8.3 Alert Governance

Mechanism	Parameter	Behaviour
Confidence gate	`0.7`	Applied in `inference.py` before the alert system is called at all.
Cooldown	`30 s`	Applied inside `trigger_alert()`, collectively across all channels. Returns `False` when suppressed.
Mute	runtime flag	Suppresses buzzer, popup and OS notification only. Console and JSON logging remain active.
Popup lifetime	`3000 ms`	Auto-dismissed; requires no operator interaction, so an unattended station is never left with a modal window.
Temporal smoothing	3 of last 5	Upstream of all the above — a fall must persist across a majority of a 5-frame window before it is even a candidate.

9. Algorithm Design

9.1 Body Orientation Angle

The primary postural indicator is the angle of the vector running from the head to the torso centre. The torso centre is the midpoint of the left and right hip keypoints, which is markedly more stable than either hip alone.

```
head   = keypoints[0][:2]                                  # nose
hip    = (keypoints[7][:2] + keypoints[8][:2]) / 2         # midpoint of L/R hips
dx, dy = hip[0] - head[0],  hip[1] - head[1]
angle  = abs( degrees( arctan2(dy, dx) ) )                 # degrees
```

Interpretation: a value near **90°** indicates an upright subject (the head-to-hip vector is vertical); values approaching **0°** or **180°** indicate a horizontal — that is, lying — posture. The absolute value collapses left-facing and right-facing orientations onto the same scale, so the measure is independent of which way the subject faces the camera.

9.2 Head Drop Speed

Vertical velocity of the head between consecutive frames, expressed in pixels per frame against the virtual 640 × 480 reference frame:

```
drop_speed = head_y(t) - head_y(t-1)        # positive = downward motion
```

A threshold of 20 px/frame corresponds to roughly 600 px/second at 30 fps — substantially faster than controlled voluntary movements such as sitting or bending, which is the discriminating property the rule exploits.

9.3 Ground Proximity

```
near_ground = head_y > 0.6 × virtual_height        # > 288 px of 480
```

When the head occupies the lower 40% of the frame, the subject is likely at or near floor level. This predicate is a necessary condition for the *lying* classification, distinguishing a genuine post-fall position from a merely stooped posture higher in the frame.

9.4 Rule-Based State Machine

```
is_lying     = (angle < 50) AND near_ground
fall_trigger = (drop_speed > 20)

_fall_counter  = _fall_counter + 1  if fall_trigger  else max(0, _fall_counter - 1)
_lying_counter = _lying_counter + 1 if is_lying      else 0

if   _fall_counter  >= 2 :  status = "fall",    confidence = min(0.6 + 0.1 × _fall_counter, 1.0)
elif _lying_counter >= 5 :  status = "warning", confidence = 0.8
elif angle < 70          :  status = "warning", confidence = 0.6
else                     :  status = "normal",  confidence = 1.0
```

The asymmetric counter treatment is deliberate: `_fall_counter` *decays* rather than resetting, so a fall spread across intermittently detected frames still accumulates, whereas `_lying_counter` resets immediately because standing up is unambiguous evidence that the subject is no longer down.

9.5 Decision Fusion

Rule and neural verdicts are combined with a default neural weight of 0.7, subject to a severity-priority override:

```
if nn_confidence == 0:                      # no model loaded / buffer not yet full
    return rule_status, rule_confidence     # 100% rule-based fallback

priority = {"normal": 0, "warning": 1, "fall": 2}

if priority[nn_status] >= priority[rule_status]:
    status     = nn_status
    confidence = 0.7 × nn_conf + 0.3 × rule_conf
elif rule_conf > 0.8:                       # rules see greater severity, and are confident
    status     = rule_status                # safety override — escalate
    confidence = 0.5 × rule_conf + 0.5 × nn_conf
else:
    status     = nn_status
    confidence = 0.7 × nn_conf + 0.3 × rule_conf
```

The middle branch encodes the central safety philosophy of the system: when the interpretable biomechanical rules detect a *more severe* situation than the network and are highly confident, the rules win. A missed fall is a far costlier error than a false alarm, so the design biases toward escalation.

9.6 Temporal Smoothing

```
history.append((status, confidence))          # bounded deque, maxlen = 10
recent = history[-5:]                         # 5-frame voting window

if count(recent, "fall")    >= 3:  return "fall",    mean_confidence("fall")
if count(recent, "warning") >= 2:  return "warning", mean_confidence("warning")
return most_frequent(recent), mean_confidence(most_frequent)
```

Thresholds are asymmetric by severity: a `fall` requires a 3-of-5 majority (suppressing spurious alarms), while a `warning` requires only 2-of-5 (surfacing early instability readily, since the cost of an unnecessary warning is low). Fewer than three frames of history pass through unmodified so the system responds immediately at start-up.

10. Non-Functional Design

10.1 Performance

Design decision	Performance rationale
YOLOv8 *nano* variant	Smallest pose model in the family; keeps per-frame inference tractable on CPU-only hardware.
13 of 17 keypoints retained	Reduces the feature vector by 24% and removes four facial landmarks that contribute noise rather than postural signal.
Bounded deque buffers	O(1) append with automatic eviction; no array reallocation per frame.
`torch.no_grad()` at inference	Disables autograd bookkeeping, reducing both latency and memory.
Threaded alert channels	The buzzer alone blocks ~600 ms; dispatching it inline would drop roughly 18 frames per alert.
Detection fused into the stream generator	One inference pass serves both the video stream and the telemetry API — no duplicated computation.
`threaded=True` Flask server	Prevents the long-lived MJPEG response from starving concurrent `/status` requests.

Observed on the development machine (CPU-only): sustained 25–30 fps end-to-end with one subject in frame, meeting the SRS target of ≥ 25 fps.

10.2 Reliability and Fault Tolerance

Failure mode	Design response
Neural checkpoint missing or corrupt	`_load_model()` catches all exceptions, logs a warning and returns `None`. The system continues in rule-only mode — degraded accuracy, uninterrupted service.
Camera read failure	Consecutive failures are counted; brief failures are retried after 100 ms, and after 15 consecutive failures the device handle is released, held for one second, and reopened. The stream is never terminated by a transient fault.
Client disconnect mid-stream	A `finally` block releases the capture handle, so the device is always available for reacquisition — important on drivers that permit only one open handle.
Subject briefly occluded	Last-known keypoints are reused for up to 10 frames, preventing loss of an in-progress fall sequence.
Audio device unavailable	`_sound_alert()` catches the exception and falls back to terminal bells.
SMTP unreachable or unconfigured	Runs on a daemon thread with a broad exception handler; failure is logged and never propagates to the detection loop.
Log file corrupted	`_load_existing_log()` catches decode errors and starts a fresh in-memory list rather than crashing at start-up.

10.3 Privacy and Security

- **No video retention.** Frames exist only in memory. The sole persisted imagery is one JPEG per confirmed alert.
- **Local-only processing.** All inference is on-device; no third-party service receives video.
- **Minimal egress.** The optional e-mail channel transmits event metadata and a single frame, and is disabled by default.
- **Auditability.** Every alert is recorded with timestamp, status, confidence and evidence frame, supporting after-the-fact review.

> **Known gap (FYP-2).** The Flask server binds to `0.0.0.0` with no authentication, so any host on the local network can reach the dashboard and video feed. This is acceptable for a controlled demonstration but must be addressed before any real deployment. Planned remediation: bind to `127.0.0.1` by default, and add session authentication plus TLS for remote access.

10.4 Maintainability

- **Single-responsibility modules** with narrow, array-based interfaces allow any stage to be replaced independently — the MediaPipe → YOLOv8 migration required changes to `pose_estimator.py` alone.
- **No magic numbers** in algorithm code; every threshold resolves to a named configuration field.
- **Train/serve parity** is structural: the training pipeline imports the very same `PoseEstimator` and `FeatureExtractor` classes used at inference, eliminating a common source of silent skew.
- **Architecture substitution** via a factory function and a configuration string, with no call-site changes.

11. Requirements Traceability Matrix

Each functional requirement from the SRS is mapped to the design element that realises it and to its implementation status in the Checkpoint-2 build.

SRS Req.	Requirement summary	Design element	SDD ref.	Status
FR-1.1	YOLOv8-Pose extracts 17 keypoints per frame	`PoseEstimator.process_frame()`	§5.2.1, §6.1	Implemented
FR-1.2	Track up to 5 persons simultaneously	`MultiPersonPoseEstimator`	§5.2.2	Partial — class implemented and demonstrated standalone; not yet integrated into the dashboard pipeline
FR-1.3	Skeleton overlay on the video stream	`_draw_custom_skeleton()`	§5.2.1	Implemented
FR-1.4	Retain last position for up to 10 occluded frames	`_handle_lost_tracking()`	§5.2.1	Implemented
FR-2.1	Body orientation angle (head → hip)	`extract_frame_features()`	§9.1	Implemented
FR-2.2	Head drop speed	`get_rule_based_status()`	§9.2	Implemented
FR-2.3	Ground proximity below 60% frame height	`near_ground` predicate	§9.3	Implemented
FR-2.4	30-frame buffer, stride 5	`FeatureExtractor.update()`	§6.3	Implemented
FR-2.5	Feature standardisation from training statistics	`TemporalFeatureProcessor`	§5.3	Partial — implemented in the training path; not applied at live inference
FR-3.1	Heuristic state machine with physics thresholds	`get_rule_based_status()`	§8.1, §9.4	Partial — the `normal` and `warning` rules conform; the `fall` rule triggers on descent speed alone, omitting the ground-proximity and angle confirmation the requirement specifies (see §12.2)
FR-3.2	CNN-LSTM temporal classification	`FallDetectorCNNLSTM`	§5.4	Implemented

FR-3.3	Fusion at 70% NN weight with rule fallback	`_fuse_predictions()`	§9.5	Implemented
FR-3.4	Temporal smoothing (3 of 5 / 2 of 5)	`_smooth_predictions()`	§9.6	Implemented
FR-4.1	Live HUD with status, telemetry and FPS	`_draw_hud()`	§5.5	Implemented
FR-4.2	Responsive dashboard grid layout	`index.html` , `style.css` , `app.js`	§7.2	Implemented
FR-5.1	Audible beep pattern on fall	`_sound_alert()`	§5.6	Implemented
FR-5.2	Dashboard visual alert overlay	`fall-overlay` + `triggerFallUI()`	§7.2	Implemented
FR-5.3	Local popup alert window	`_visual_alert()`	§5.6	Implemented — duration now configurable
FR-5.4	SMTP e-mail with JPEG attachment	`_email_alert()`	§5.6	Implemented but disabled by default; requires credential configuration
FR-5.5	Configurable alert cooldown (30 s)	`trigger_alert()` gate	§8.3	Implemented
FR-5.6	Persistent daily JSON event logging	`_log_event()`	§6.4	Implemented
Beyond SRS — added during Checkpoint-2		`_desktop_notification()`	§5.6	Native OS notification channel
Beyond SRS — added during Checkpoint-2		`/alerts/mute` , camera reconnect	§7.1, §10.2	Operator mute control and camera recovery

12. Implementation Status and FYP-2 Roadmap

12.1 Delivered in the Checkpoint-2 MVP

- End-to-end real-time pipeline from webcam to classified verdict, running at 25–30 fps on CPU.
- YOLOv8-Pose estimation with skeleton overlay and occlusion tolerance.
- Biomechanical feature engineering with a fully explainable rule engine.
- Trained CNN-LSTM checkpoint loading and participating in fused decisions.
- Six alert channels with cooldown governance and an operator mute control.
- Web dashboard with live MJPEG video, real-time telemetry, connection-state indicators and an event feed.

- Persistent JSON event logging with JPEG evidence capture.
- Camera disconnect detection and automatic recovery.

12.2 Known Limitations

> **Rule-engine coupling defect (highest priority).** In the current state machine the `fall` verdict is reachable *only* through the `drop_speed > 20` trigger, while the `is_lying` predicate — which is the condition that actually encodes body angle and ground proximity — can escalate no further than `warning`. Two consequences follow: a rapid head movement near the camera can register a `fall` while the subject is upright; and a subject who is already motionless on the floor produces no velocity spike and therefore never reaches `fall`. The remediation is to require a confirmed lying posture sustained across several frames as a necessary condition for `fall`, with velocity acting as a corroborating rather than sole trigger. This is scheduled as the first task of FYP-2.

Limitation	Impact and planned resolution
Hip keypoint confidence not validated	When hips are outside the frame or poorly visible, YOLO still emits an estimated position which feeds the angle calculation unchecked, producing spurious `warning` states at close camera range. Resolution: gate the angle computation on per-keypoint confidence and hold the previous value when unreliable.
Multi-person not wired into the dashboard	The live pipeline processes only the highest-scoring subject. Resolution: per-person feature buffers and independent state machines keyed by a tracking ID.
Inference-time normalisation not applied	Training standardises features; the live path does not, introducing potential train/serve skew. Resolution: persist normalisation statistics in the checkpoint and apply them in `_predict()`.
No automated test suite	No regression protection. Resolution: unit tests for the geometric functions, the state machine and the fusion logic, plus a recorded-clip integration test.
Unauthenticated network exposure	See Section 10.3. Resolution: loopback binding by default plus authentication.
Event Log and Settings views not implemented	Navigation entries are visibly disabled. Resolution: implement in FYP-2 against the existing JSON logs and configuration model.

12.3 FYP-2 Roadmap

Priority	Work item	Description
1	Rule-engine correction	Restructure the state machine so posture and ground proximity are necessary conditions for a fall verdict; add keypoint-confidence gating.
2	Dataset and model maturation	Train on a public fall-detection corpus (URFD / Le2i) plus locally recorded scenarios; report precision, recall and F1 per class with a confusion matrix.
3	Multi-person integration	Bring `MultiPersonPoseEstimator` into the live pipeline with per-subject state and identity-tagged alerts.
4	Event Log and Settings UI	Browsable, filterable incident history with capture thumbnails; live threshold tuning from the browser.
5	Alert channel expansion	SMS/WhatsApp gateway integration for off-site caregivers.
6	Security hardening	Authentication, TLS, and secure-by-default network binding.
7	Test automation	Unit and integration suites with continuous execution on push.
8	Edge optimisation	ONNX Runtime inference path; feasibility assessment on Raspberry Pi / Jetson Nano class hardware.

13. Appendix A — Configuration Parameter Reference

Complete enumeration of tunable parameters as defined in `config.py`.

A.1 Pose and Feature Parameters

Parameter	Default	Effect
`min_detection_confidence`	0.5	Minimum YOLO score for a person detection to be accepted.
`min_tracking_confidence`	0.5	Minimum score for continued tracking.
`CRITICAL_KEYPOINTS`	13 indices	Subset of YOLO joints retained for analysis.
`sequence_length`	30	Frames per temporal window (~1 s at 30 fps).
`stride`	5	Sliding-window step during offline sequence generation.
`num_raw_features`	39	13 keypoints × 3 values.
`num_engineered_features`	12	5 active + 7 reserved.
`vertical_speed_threshold`	0.15	Normalised rapid-descent threshold (training-side).
`body_angle_threshold`	45.0°	Reference tilt threshold.
`stillness_threshold`	0.005	Post-fall motion floor.
`stillness_duration_frames`	15	Frames of stillness confirming a fall.

A.2 Model and Training Parameters

Parameter	Default	Effect
`architecture`	`cnn_lstm`	Selects `lstm`, `cnn_lstm` or `transformer`.
`input_size`	51	Feature dimensions per frame.
`hidden_size`	128	LSTM hidden units / Transformer model width.
`num_layers`	2	Recurrent depth.
`num_classes`	3	normal, warning, fall.
`dropout`	0.3	Regularisation strength.
`bidirectional`	true	Doubles effective LSTM output width.
`cnn_filters`	[64, 128]	1-D convolution channel progression.
`batch_size` / `learning_rate`	32 / 0.001	Optimiser configuration.
`epochs` / `patience`	100 / 15	Training length and early-stopping window.
`class_weights`	[1.0, 2.0, 3.0]	Loss weighting; penalises missed falls most heavily.

A.3 Runtime and Alert Parameters

Parameter	Default	Effect
`camera.source`	0	Device index or media path / URL.
`camera.width` × `height`	640 × 480	Requested capture resolution.
`device`	`auto`	Resolves to CUDA when available, otherwise CPU.
`confidence_threshold`	0.7	Minimum fused confidence to raise an alert.
`enable_sound` / `enable_email` / `enable_logging`	true / false / true	Per-channel enablement.
`alert_cooldown_seconds`	30.0	Minimum interval between alerts, all channels.
`popup_duration_ms`	3000	On-screen popup lifetime.
`log_dir`	`logs`	Root for event logs and captures.

A.4 Repository Structure

```
FYP-2026-G6-RealTimeAutomatedFallDetectionAndAlertSystem/
├── docs/           SRS, this Software Design Document
├── architecture/   Architecture diagrams (Checkpoint-2)
├── src/            Application source code
├── test/           Test scenarios and results
├── evidence/       Implementation evidence — screenshots, logs, demo capture
├── dataset/        Dataset references and preparation notes
├── research/       Literature review and background reading
├── reports/        Progress and final reports
├── presentations/  Proposal and defence slide decks
└── meeting_logs/   Supervisor meeting records
```

End of Software Design Document

GuardianAI — Real-Time Automated Fall Detection and Alert System · FYP-2026-G6 · Version 1.0

Prepared for FYP-I Checkpoint-2 · Subject to Faculty Supervisor review and approval